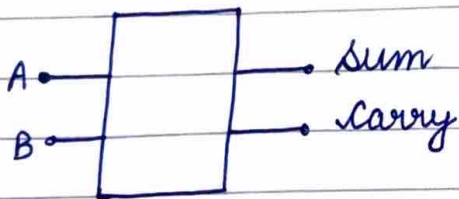


→ Comparison between combinational & sequential circuit
 Combinational circuits the output is only dependent on present input whereas sequential ckts require present as well as previous output.

Combinational → Adder, Subtractor, Mix, Demux
 Sequential → Flip Flop

→ Half Adder



It's used to add single bit numbers.

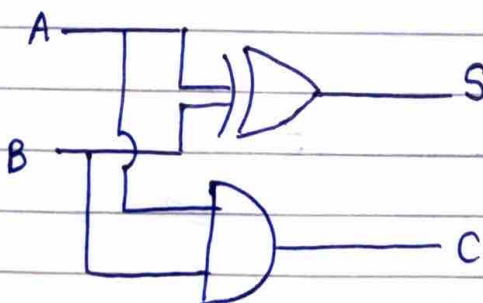
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

	$\bar{A}$	A
$\bar{B}$	0	1
B	1	0

$$S = \bar{A}B + A\bar{B}$$

$$S = A \oplus B$$

$$C = A \cdot B$$



→ Full Adder

A	B	Cin	S	Count
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

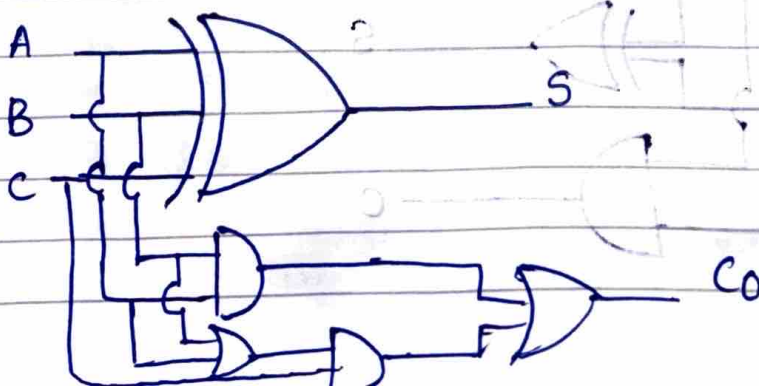
	$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
$\bar{C}$	0	1	0	1
C	1	0	1	0

$$\begin{aligned}
 S &= \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}B\bar{C} \\
 &= B(\bar{A}\bar{C} + AC) + \bar{B}(\bar{A}\bar{C} + \bar{A}C) \\
 &= B(A \oplus C) + \bar{B}(A \oplus C) \\
 &= B(\bar{A} \oplus C) + \bar{B}(A \oplus C) \\
 &= B \oplus A \oplus C
 \end{aligned}$$

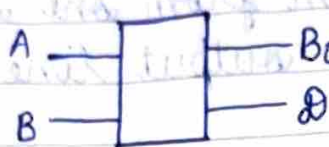
	$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
$\bar{C}$	0	0	1	0
C	0	1	1	1

$$C_0 = AB + BC + AC$$

$$C_0 = AB + C(A+B)$$



→ Half Subtractor



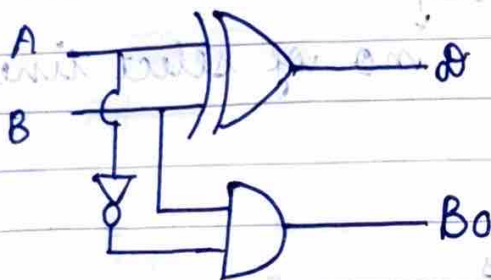
0	1
1	0

$$D = A \oplus B$$

A	B	B ₀	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Y = 11 ly

$$B_0 = \bar{A}B$$



→ Full Subtractor

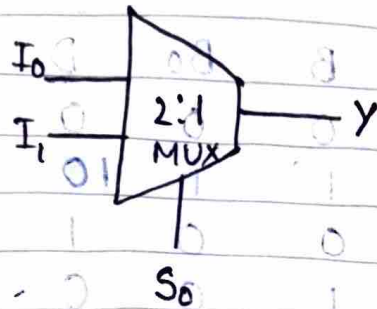
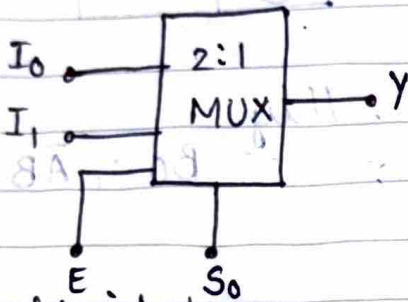
A	B	Bin	D	B ₀
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = A \oplus B \oplus C$$

$$B_0 = B \text{Bin} + \bar{A}(B + \text{Bin})$$

→ Multiplexers

It selects binary information from one of many inputs and directs it to output line using select lines.



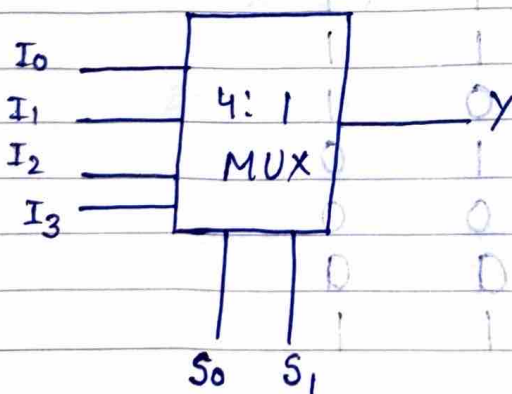
enable input
($E=0$ so $Y=0$ always
chahi input kuch bhi)

$n = 2^m$
no. of inputs no. of select lines

• Advantages of MUX

- Reduces no. of wires
- Reduces ckt complexity & cost
- Various ckts can be implemented

→ 4:1 MUX



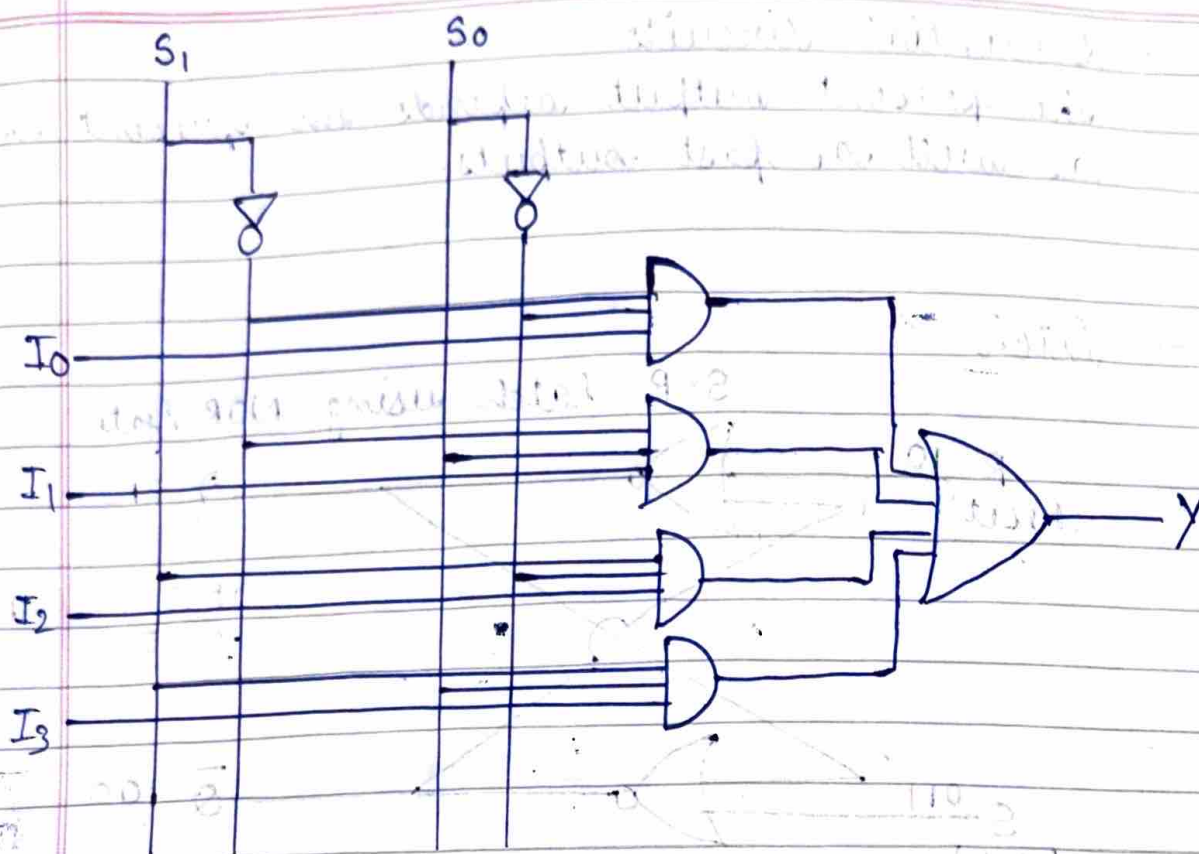
For $E=0$, $Y=0$

For $E=1$

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

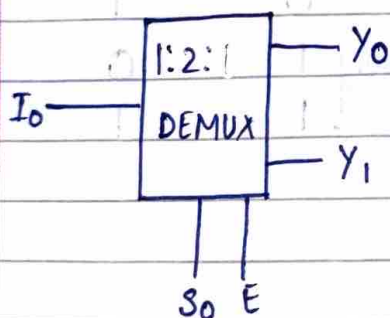
$$Y = A \oplus B \oplus C$$

$$(a \oplus b + a) \bar{A} + a \oplus b = 0$$



→ Demultiplexer

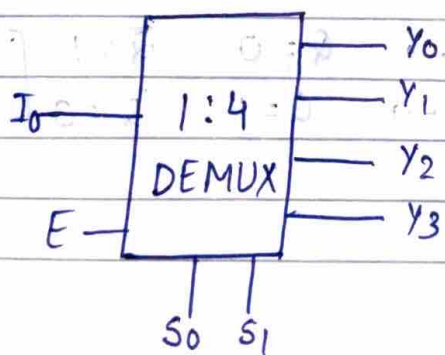
- one input to give many outputs
- used to distribute data



E	S	Y0	Y1
0	X	0	0
1	0	1	0
1	1	0	1

$$Y_0 = E \bar{S}_0 I_0$$

$$Y_1 = E S_0 I_0 = Y$$

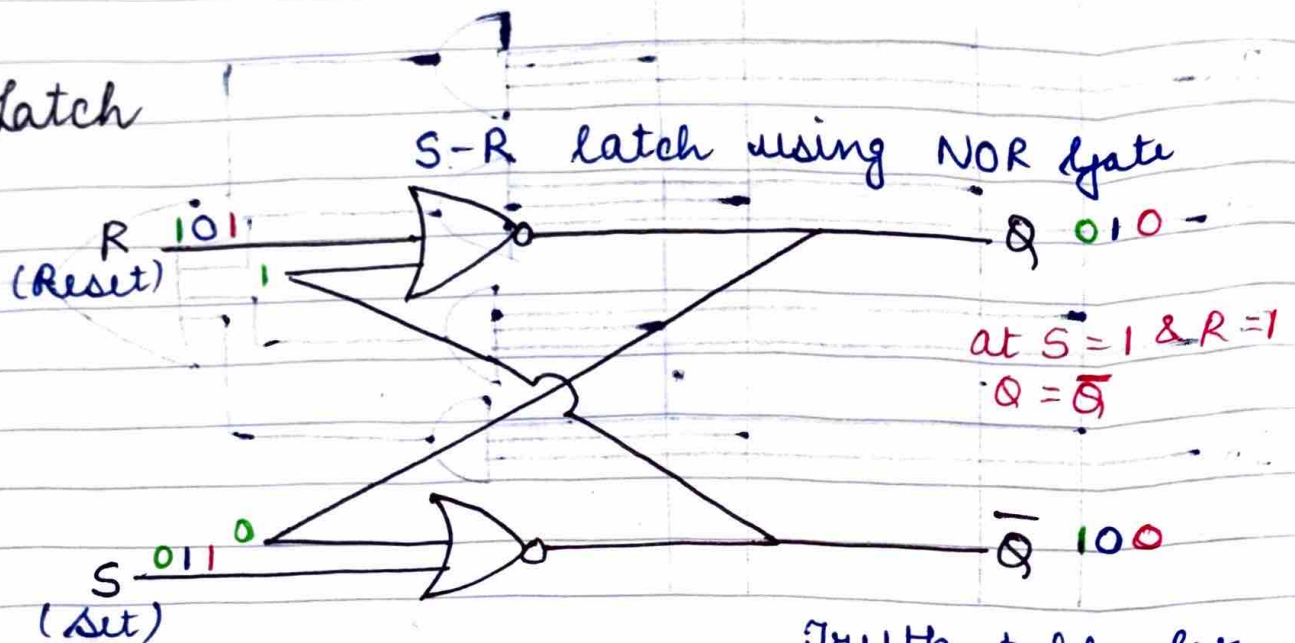


E	S0	S1	Y0	Y1	Y2	Y3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

→ Sequential Circuits

The present output depends on present input as well as past outputs.

→ Latch



Reset $\rightarrow Q = 0$

Set $\rightarrow Q = 1$

Truth table for NOR gate

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Jab ek input 1 hai to $Y=0$
 $\Rightarrow Q=0$ hoga

Case 1 $S=0$ & $R=1$

$Q=0$ & $\bar{Q}=1$

Case 2 $S=1$ & $R=0$

$Q=1$ & $\bar{Q}=0$

Case 3 $S=0$ & $R=0$

If we see after Case 1 $\rightarrow Q=0$ & $\bar{Q}=1$ [Memory]

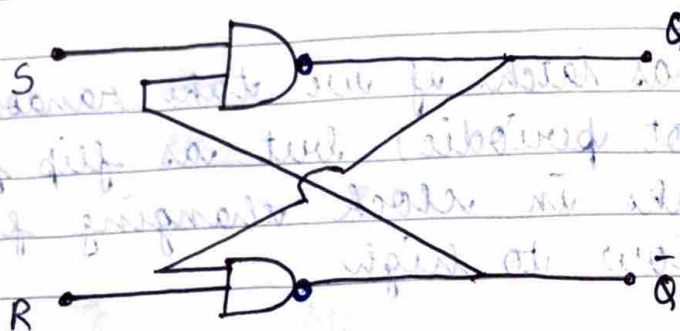
If we see after Case 2 $\rightarrow Q=1$ & $\bar{Q}=0$

Case 4 $S=1$ & $R=1$

Not used because $Q = \bar{Q}$

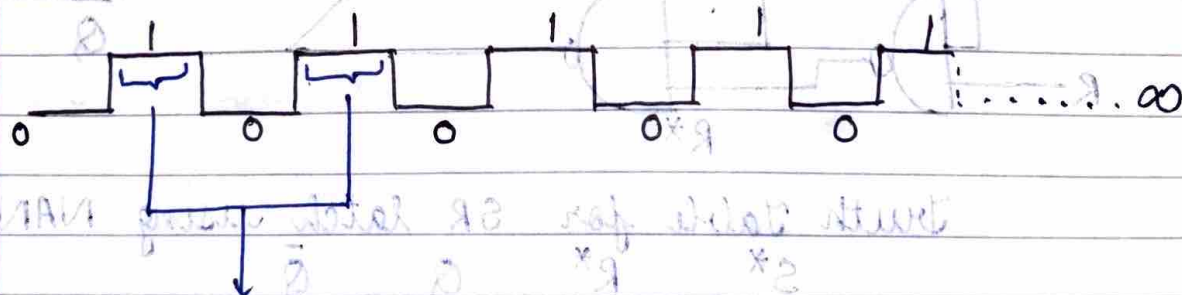
S	R	Q	$\bar{Q}$
0	0	Memory	
0	1	0	1
1	0	1	0
1	1	Not used	

→ SR latch using NAND gate



S	R	Q	$\bar{Q}$
0	0	NOT USED	
0	1	1	0
1	0	0	1
1	1	MEMORY	

→ clock

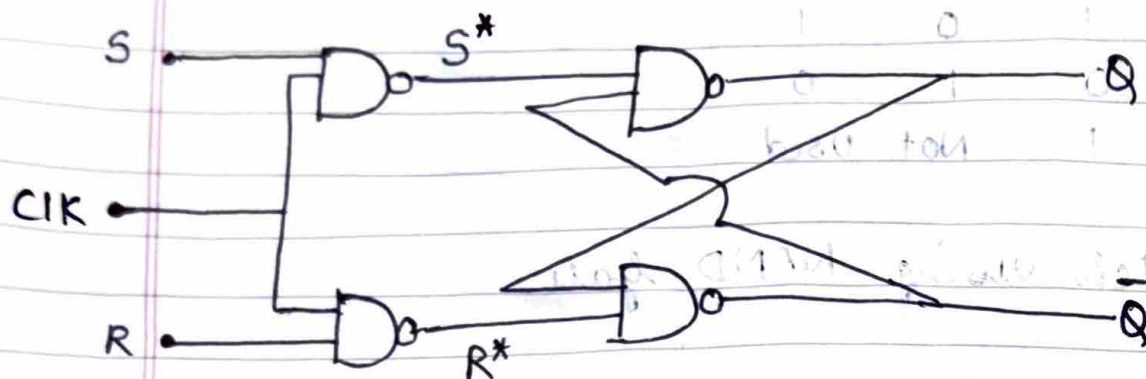


Flip Flop
functions when

clk → 1

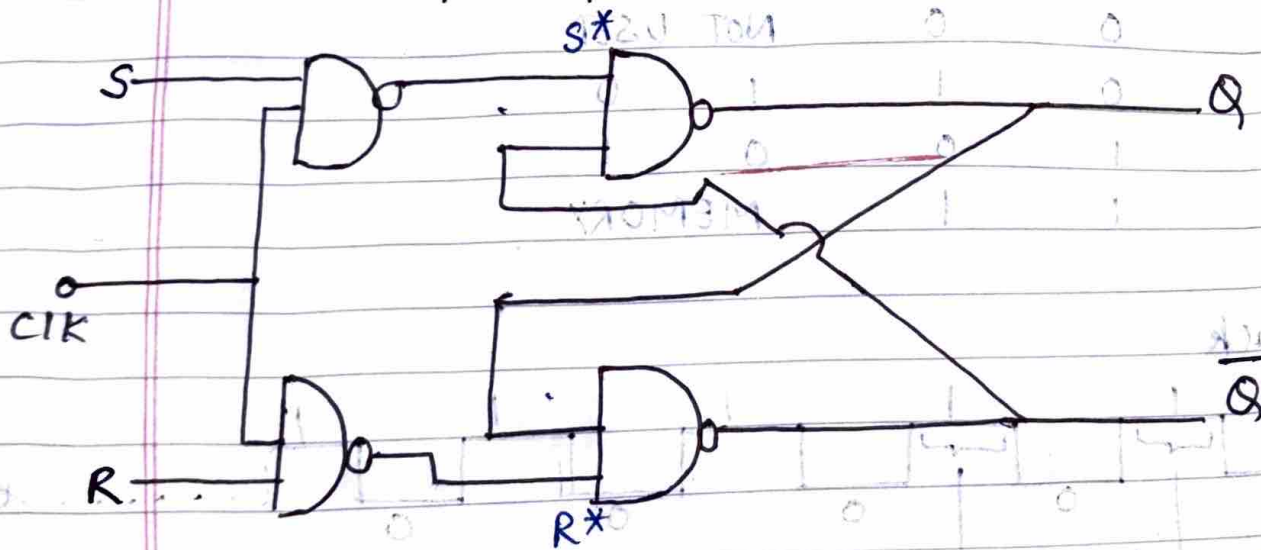
to avoid random
changing of outputs

→ Difference between Latch and flip flops



It behaves as latch if we take random enable signal (not periodic) but as flip flop it only works in clock changing from high to low / low to high

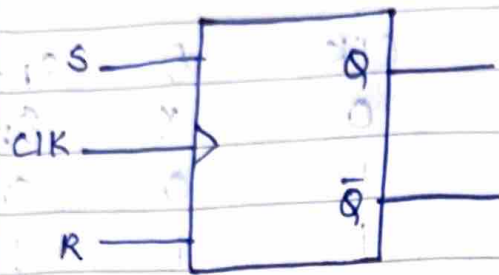
→ S-R Flip Flop



Truth Table for SR latch using NAND

	S^*	R^*	Q	$\bar{Q}$
$S^* = \bar{S} + \bar{CLK}$	0	0	Not used	
$R^* = \bar{R} + \bar{CLK}$	0	1	1	0
	1	0	0	1
	1	1	Memory	

structure for project

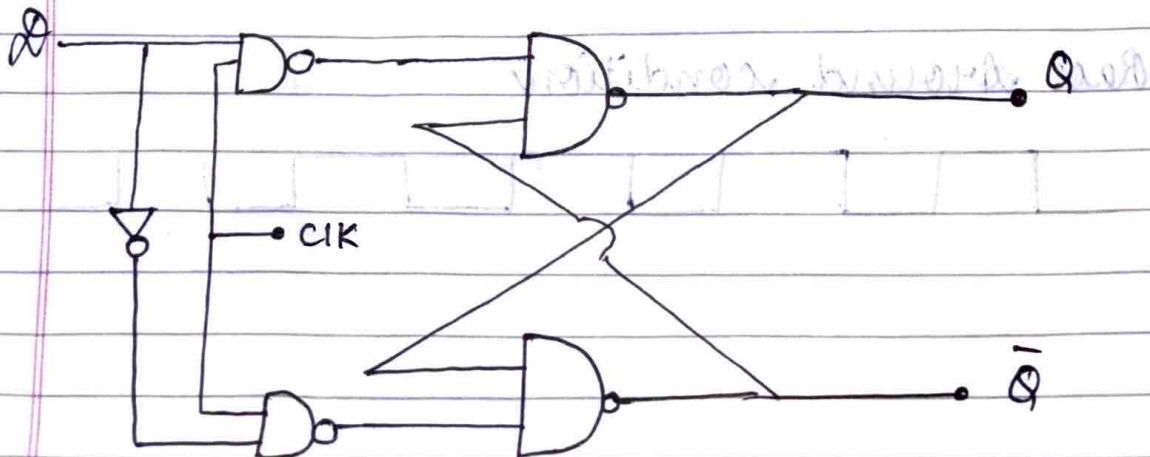


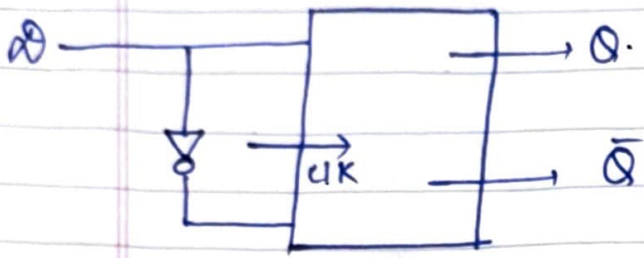
CLK	S	R	Q	$\bar{Q}$
0	X	X	Memory	Memory
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	Not used	Not used

For Q_{n+1} , Memory = Q_n

CLK	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Invalid

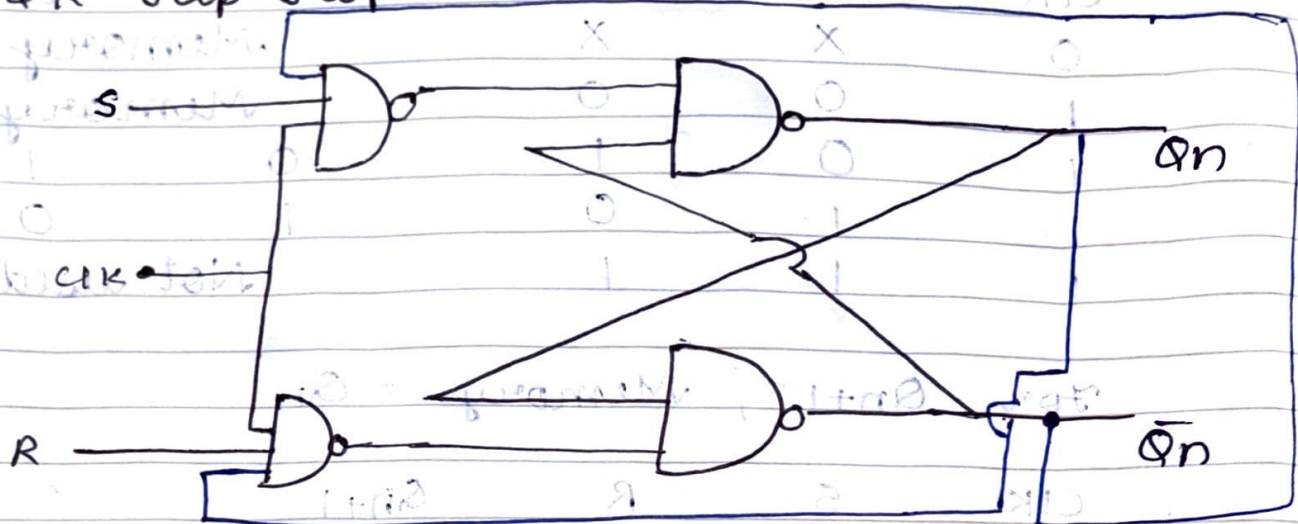
→ $\otimes$ Flip Flop (Data Flip Flop)





clk	S	R	Q_{n+1}	Q_n
0	X	X	Q_n	Q_n
1	0	0	0	0
1	1	1	1	1

→ J & K Flip Flop



clk	J	K	Q_{n+1}	Q_n
0	X	X	Q_n	Q_n
1	0	0	Q_n	Q_n
1	0	1	0	1
1	1	0	1	0
1	1	1	$\bar{Q}_n$	Q_n

→ Race Around condition



J=1

K=1

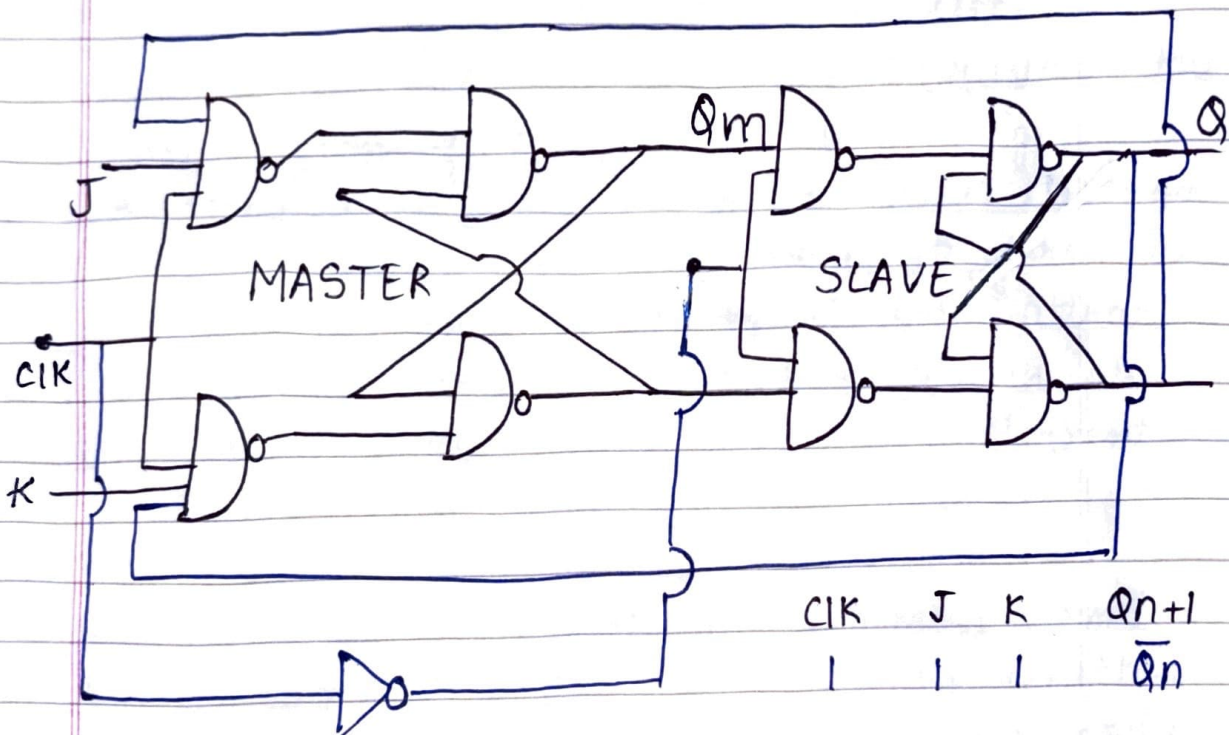


↳ Race around condition

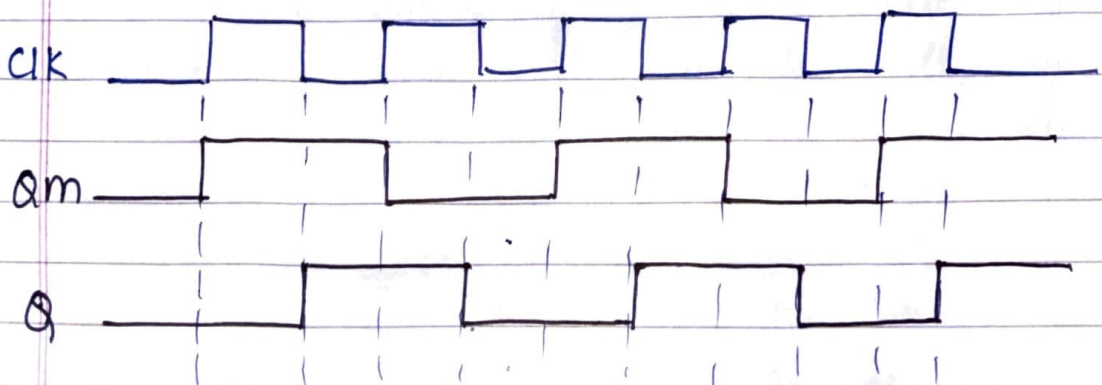
→ A race around condition is a phenomena that occurs when output of a flip flop toggles multiple times within a single clock pulse leading to an unpredictable final state. To prevent it :-

- we can use edge triggering
- we can use master slave J-K configuration
- we can control delay.

→ Master slave J-K flip flop → -ve edge triggered

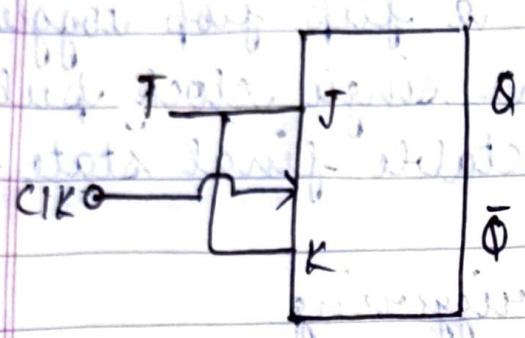


clock high, master works, slave doesn't
clock low, slave works, master doesn't



Output changes once in a cycle

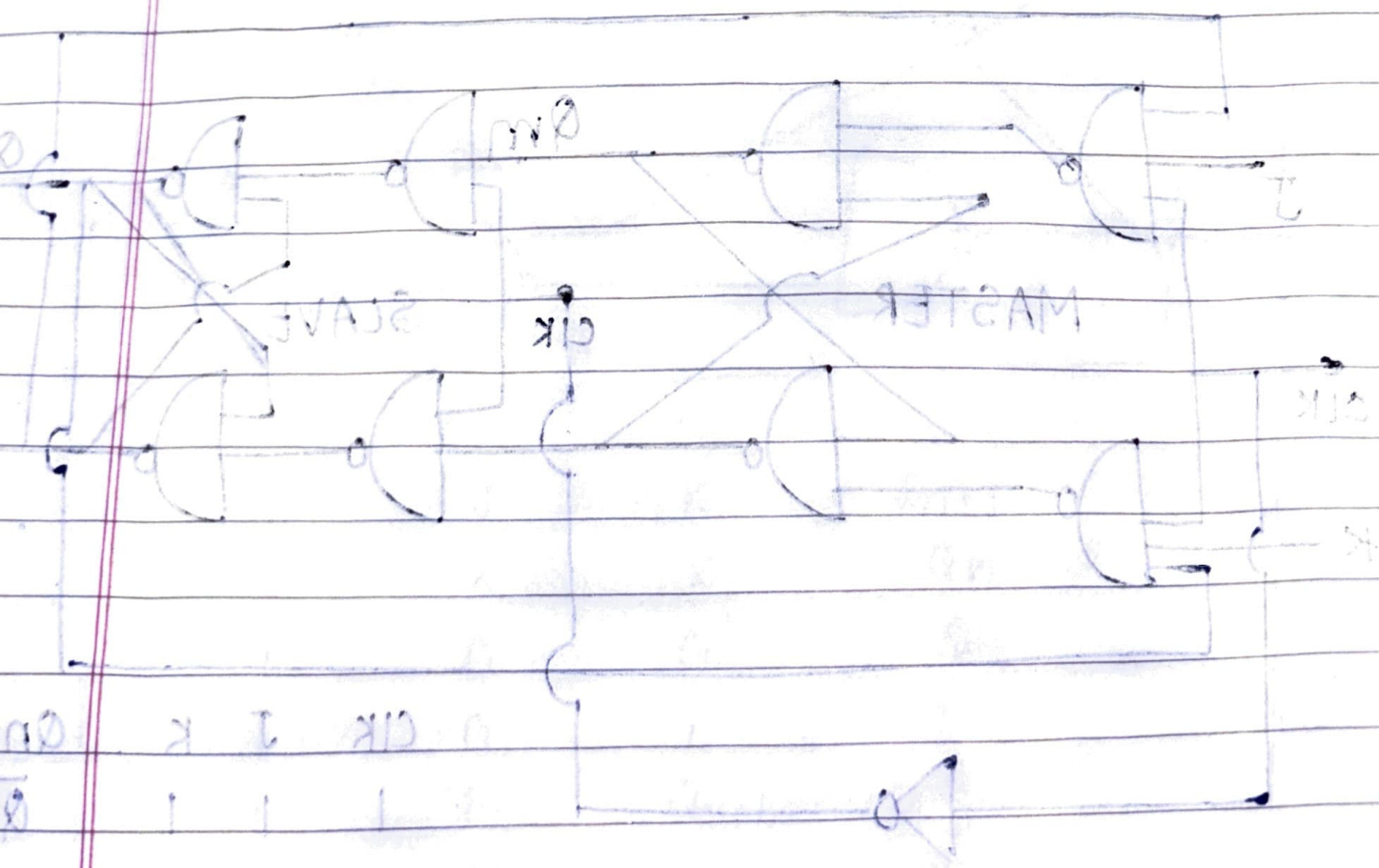
→ J Flip Flop is universal because we can



CLK	J	K	Q _{n+1}
0	X	X	Q _n
1	0	0	0
1	1	1	Q _n

Master-Slave J-K flip-flop is used to avoid race condition. It consists of two J-K flip-flops connected in series. The output of the first flip-flop is connected to the input of the second flip-flop. The clock input of the second flip-flop is connected to the output of the first flip-flop.

Master-Slave J-K flip-flop is used to avoid race condition. It consists of two J-K flip-flops connected in series. The output of the first flip-flop is connected to the input of the second flip-flop. The clock input of the second flip-flop is connected to the output of the first flip-flop.



Master-Slave J-K flip-flop is used to avoid race condition. It consists of two J-K flip-flops connected in series. The output of the first flip-flop is connected to the input of the second flip-flop. The clock input of the second flip-flop is connected to the output of the first flip-flop.

